

Synopsis

Clinical Risk Assessment for Type-2 Diabetes

Problem Id:07

Student Name- Pratiksha Raj

University Roll number- 2028467

Section-K

Student Name- Vanshika

University Roll number- 2028899

Section-K

Student Name- Anshika Srinet

University Roll number - 2028393

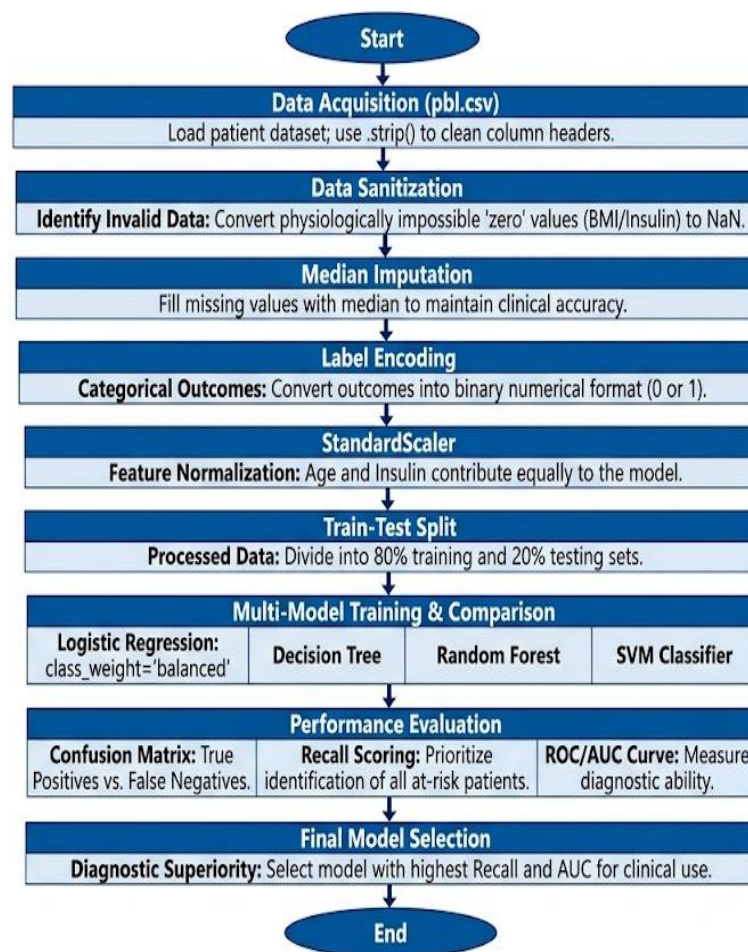
Section-K

- **Problem Description:**

Type-2 Diabetes is one of the most common chronic diseases worldwide, and early prediction is critical in preventing severe complications such as heart disease, kidney failure, and nerve damage. The primary objective of this project is to develop a machine learning model that predicts the clinical risk of Type-2 Diabetes based on key health parameters: Age, Body Mass Index (BMI), and Insulin Levels.

- **Data Acquisition & Cleaning:** The project utilizes a clinical dataset (pbl.csv) to simulate real patient scenarios. To ensure data integrity, the system cleans column headers and identifies physiologically impossible "zero" values in BMI and Insulin. These are handled via Median Imputation, ensuring the model is not skewed by missing data.
- **Preprocessing & Normalization:** Categorical risk outcomes are converted into numerical form using Label Encoding. Furthermore, StandardScaler is applied to normalize the data. This is essential because parameters like Insulin often have much higher numerical ranges than Age; scaling ensures all features contribute equally to the final prediction.
- **Comparative Modeling:** While the core logic utilizes a Logistic Regression classifier with "balanced" class weights, the project further extends to a comparative analysis of Decision Trees, Random Forests, and SVM. This allows for the selection of the most robust model for health diagnostics.
- **Evaluation Strategy:** Performance is measured using a Confusion Matrix and ROC Curves. We place a strategic focus on the Recall Score, as correctly identifying at-risk patients (avoiding False Negatives) is more vital in medical applications than minimizing false alarms. Ultimately, this project demonstrates how machine learning serves as a reliable decision support system for healthcare professionals.

- **Steps of Implementation:**



Steps of Preprocessing:

- Loading and Cleaning the Dataset** The dataset is loaded using `pd.read_csv()` and column names are stripped of extra whitespace using `str.strip()`. The dataset contains 8,000 rows and 9 columns including Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, and Outcome.
- Replacing Zero Values with NaN** Physiologically impossible zero values in columns like Glucose, BloodPressure, SkinThickness, Insulin, BMI, and Age are replaced with NaN. A zero reading in these medical columns represents missing or invalid data, not an actual measurement.
- Handling Invalid and Negative Values** Any value that is medically invalid — such as Age less than or equal to zero, negative BMI, or negative Glucose — is set to NaN. A loop across all numeric columns also converts any remaining negative values to NaN to ensure data integrity.
- Missing Value Treatment** All NaN values are filled using the median of each respective column using `fillna(df.median())`. Median is preferred over mean in medical datasets as it is robust to extreme values and skewed distributions.

e) **Outlier Capping using IQR** Instead of removing outlier rows, a custom `cap_outliers()` function is applied to clip values to the IQR boundary ($Q1 - 1.5 \times IQR$ to $Q3 + 1.5 \times IQR$) for columns Age, BMI, Glucose, and Insulin. Capping preserves all 8,000 rows while removing the distorting effect of extreme values.

- **Steps of Feature Selection:**

Using All Features All eight input columns — Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, and Age — are used as features. No feature is dropped as each represents a clinically relevant health parameter that contributes to diabetes risk prediction.

- **Steps of Train-Test Split and Scaling:**

- a) **Train-Test Split (80-20)** The dataset is split into 80% training data and 20% test data using `train_test_split` with `stratify=y` to maintain the same class ratio (66.3% Non-Diabetic, 33.7% Diabetic) in both subsets.
- b) **StandardScaler Normalization** `StandardScaler` is fitted only on training data and applied to both training and test sets. This prevents data leakage and ensures that features with large numerical ranges like Insulin do not dominate features with smaller ranges like Age. All features are normalized to zero mean and unit variance.
- c) **Class Imbalance Handling** The dataset has a class imbalance — 5,302 Non-Diabetic vs 2,698 Diabetic samples. To handle this, `class_weight='balanced'` is applied to Logistic Regression and SVM so that minority class predictions are not suppressed during training.

- **Steps of Model Training:**

- a) **Logistic Regression (LR)** Logistic Regression is a linear classifier trained with `class_weight='balanced'` and `max_iter=1000`. It acts as the baseline model. It achieved a validation accuracy of 73.31%.
- b) **Decision Tree (DT)** A Decision Tree classifier with `max_depth=5` is used to capture non-linear boundaries in the data. It achieved a validation accuracy of 81.06%.
- c) **Random Forest (RF)** Random Forest with `max_depth=5` is an ensemble of multiple decision trees that reduces overfitting through aggregation. It achieved a validation accuracy of 82.25%.
- d) **Gradient Boosting (GB)** Gradient Boosting with 100 estimators and learning rate 0.1 builds trees sequentially, each correcting the errors of the previous one. It achieved the highest base validation accuracy of 87.88%.

e) Support Vector Machine (SVM) SVM with an RBF kernel finds an optimal margin-based hyperplane to separate diabetic and non-diabetic classes. It achieved a validation accuracy of 83.56%.

f) Ensemble Voting Classifier All five models are combined using a soft-voting VotingClassifier. It averages the predicted probabilities from all five models and selects the class with the highest combined score, achieving a validation accuracy of 84.31%.

- **Steps of Hyperparameter Tuning:**

a) GridSearchCV GridSearchCV with 3-fold cross-validation is applied to all five individual models, optimizing for Recall score since correctly identifying diabetic patients (avoiding False Negatives) is the clinical priority.

b) Tuned Results After tuning, the performance improved significantly across all models:

- Logistic Regression Tuned: Accuracy 75.75%, Recall 52.59%
- Decision Tree Tuned: Accuracy 89.19%, Recall 81.67%
- Gradient Boosting Tuned: Accuracy 89.94%, Recall 82.78%
- SVM Tuned: Accuracy 85.75%, Recall 78.15%
- Random Forest Tuned: Accuracy 94.31%, Recall 91.30% (best)

b) Final Model Selection Tuned Random Forest is selected as the final model with the highest accuracy of 94.31% and recall of 91.30%, outperforming all other tuned and base models across every metric.

- **Steps of Final Evaluation:**

a) Confusion Matrix The confusion matrix is plotted using Seaborn heatmap to visualize True Positives, True Negatives, False Positives, and False Negatives. In a medical context, False Negatives (missed diabetic patients) are the most critical error and minimizing them was the core objective.

b) ROC Curve The ROC curve is plotted for the final Random Forest Tuned model showing the trade-off between True Positive Rate and False Positive Rate. A higher AUC confirms strong discriminative ability of the model to distinguish diabetic from non-diabetic patients.

c) Model Saving The final trained Random Forest model and the fitted StandardScaler are saved using pickle as diabetes_model.pkl and scaler.pkl respectively, enabling the model to be reused for real-time prediction without retraining.

d) Prediction System A standalone prediction script loads the saved model and scaler, accepts patient health inputs at runtime, scales them, and outputs whether the patient is Diabetic or Not Diabetic along with the probability score.

- **Expected Modules/ Libraries to be used :**

1. **NumPy**
Used for numerical operations and generating synthetic numerical health data.
2. **Pandas**
Used for dataset creation, data manipulation, and structured data handling.
3. **Scikit-learn(sklearn)**
Used for preprocessing (LabelEncoder, StandardScaler), Logistic Regression model, and evaluation metrics.
4. **MatplotlibSeaborn**
Used for plotting graphs and Confusion Matrix visualization.
5. **Random**
Used for generating synthetic data within medical ranges.

- **Expected Evaluation metrics and Plots to be used:**

- **ConfusionMatrix**
The Confusion Matrix will be used to evaluate the classification performance by showing True Positive, True Negative, False Positive, and False Negative predictions. It helps in understanding how well the model distinguishes between risk and non-risk patients.
- **RecallScore**
Recall is important in this project because it measures how many actual diabetic risk patients are correctly identified. In medical applications, missing a positive case can be dangerous, so Recall is an important metric.
- **AccuracyScore**
Accuracy will be used to measure the overall correctness of the model by calculating the ratio of correct predictions to total predictions.
- **HeatmapPlot**
A heatmap visualization of the Confusion Matrix will be used to make the evaluation more interpretable and visually clear.
- **ROC Curve** A plot used to measure the area under the curve for model reliability.
- **FeatureDistributionPlot**
Basic plots such as histograms may be used to visualize the distribution of Age, BMI, and Insulin levels in the synthetic dataset.

- **Required topics from the subject:**

- **Arrays**
Arrays are used to store multiple values of health parameters such as Age, BMI, and Insulin levels. In machine learning, datasets are internally handled as arrays or matrices.
- **Strings**
Strings are used to handle categorical data such as risk labels (High Risk / Low Risk). These values are later converted into numerical format using encoding techniques.

- **FileHandling**
File handling concepts are useful for reading and storing datasets in CSV format. It allows saving the generated synthetic data and reusing it for model training and testing.
- **Functions**
Functions help in modular programming by separating different stages such as data preprocessing, model training, and evaluation. This improves code readability and reusability.
- **Libraries/Modules**
Understanding libraries is important because machine learning depends on external modules like NumPy, Pandas, and sklearn which provide predefined functions for efficient implementation.
- **Platform Required:**
Visual studio
- **Books and Link Sources:**
Online Link Sources:
 1. Scikit-learnDocumentation
<https://scikit-learn.org>
 2. PandasDocumentation
<https://pandas.pydata.org>
 3. NumPyDocumentation
<https://numpy.org>
 4. MatplotlibDocumentation
<https://matplotlib.org>